

eyJhbGciOiJIUzI1Ni.
eyJzdWIiOjEsIm5vbW.
jJAKmJTRGcHR3lPheY

JSON Web Token (JWS) y autenticación en dos pasos para una API back-end

Desarrollo Full Stack · Unidad 3: Desarrollo FullStack · Sesión de 90 minutos

Facultad de Ingeniería de Sistemas

Ruta de la sesión: 90 minutos



Inicio

0–10'

Caso detonante y saberes previos

Utilidad

10–15'

Dónde se usa JWT y 2FA

Transformación

15–55'

JWT: teoría, código y resultados · 2FA con TOTP

Práctica

55–85'

Laboratorio guiado con el proyecto demo

Cierre

85–90'

Retroalimentación y metacognición

Logro de la sesión: el estudiante protege una API REST con JWT firmado (JWS), aplica autorización por roles y agrega un segundo factor TOTP, verificando los resultados con un cliente HTTP.

Caso: “Alguien entró a mi cuenta”

INICIO · 0–10'



Lunes, 8:00 a. m.

Una alumna reporta que alguien ingresó al campus virtual con su contraseña, filtrada en otra web donde la reutilizaba. El atacante descargó sus notas y cambió su correo de recuperación.

La contraseña era correcta. El sistema hizo exactamente lo que se le programó.

Preguntas para discutir

1. HTTP no tiene estado: ¿cómo “recuerda” la API que ya iniciaste sesión?
2. ¿Basta con una contraseña para demostrar quién eres?
3. ¿Qué debió pedir el sistema además de la contraseña?

Saberes previos: tres preguntas rápidas

INICIO · 0–10'



¿Qué código HTTP indica “no autenticado” y cuál “sin permiso”?

401 Unauthorized · 403 Forbidden



¿En qué parte de la petición viaja una credencial como un token?

En el header Authorization: Bearer <token>



¿Por qué no se guarda la contraseña “tal cual” en la base de datos?

Se guarda un hash con sal (bcrypt), irreversible

¿Dónde lo vas a usar?

UTILIDAD · 10–15'



Banca y billeteras móviles

La app guarda un token de corta duración y pide un código o biometría para operaciones sensibles.



Iniciar sesión con Google

OpenID Connect entrega un id_token que es un JWT firmado por el proveedor.



Microservicios

Cada servicio valida el JWT sin consultar la base de usuarios: arquitectura sin estado y escalable.



Tu empleabilidad

Autenticación con JWT y MFA aparece como requisito habitual en puestos de back-end y full stack.

Parte 1 · JSON Web Token

Autenticación sin estado, firma digital y autorización por roles · 15–35 min



Autenticación vs. autorización

TRANSF. · 15–55'



Autenticación

¿Quién eres?

Verifica la identidad: contraseña, token, código 2FA.

401 Unauthorized

Sin token, token falso o vencido



Autorización

¿Qué puedes hacer?

Verifica permisos sobre un recurso: roles, scopes.

403 Forbidden

Identidad válida, pero sin permiso

Sesión en servidor vs. token sin estado

TRANSF. · 15–55'



Sesión tradicional (stateful)

- El servidor guarda la sesión en memoria o en BD
- El navegador solo lleva un session_id en una cookie
- Escalar a varios servidores exige compartir sesiones (Redis)
- Cerrar sesión es inmediato: se borra en el servidor



Token JWT (stateless)

- El servidor no guarda nada: el token contiene los datos
- El cliente envía el token en cada petición
- Cualquier instancia valida la firma con la clave
- Revocar antes de exp requiere lista negra o tokens cortos

JWT, JWS y JWE: ¿cuál es cuál?

TRANSF. · 15–55'

JWT RFC 7519

Formato estándar para transportar “claims” (afirmaciones) en JSON compacto y seguro para URLs.

El contenedor

JWS RFC 7515

JWT firmado digitalmente. Garantiza integridad y autenticidad: nadie puede modificarlo sin la clave.

Lo que usaremos hoy

JWE RFC 7516

JWT cifrado. Garantiza confidencialidad: el contenido no se puede leer sin la clave.

Casos especiales



Firmado no es cifrado. Un JWS se codifica en Base64URL: cualquiera puede leer su contenido. Nunca pongas contraseñas ni datos sensibles en el payload.

Anatomía de un JWT real de nuestra API

TRANSF. · 15–55'

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEsIm5vbWJyZSI6IkFuYSIsInJvbCI6ImFkbWluIiwiYW1y
IjpbInB3ZCJdLCJpYXQiOjE3OTM3NDIzMzcsImV4cCI6MTc5Mzc0MzIzNywiaXNzIjoiYXBpLWZ1bGxzdGFjayJ9.jJAK
mJTRGcHR3lPheYkxVgUVsAEoRooHB70lGSNSJgI
```

HEADER

Algoritmo y tipo

```
{
 "alg": "HS256",
 "typ": "JWT"
}
```

PAYLOAD

Claims (datos)

```
{ "sub": 1, "nombre": "Ana",
  "rol": "admin", "amr": ["pwd"],
  "iat": 1793742337,
  "exp": 1793743237,
  "iss": "api-fullstack" }
```

SIGNATURE

Firma HMAC-SHA256

```
jJAKmJTRGcHR3lPheYkxVgUVsA
…
```

Verifica que header y payload no fueron alterados

Claims: lo que el token afirma

TRANSF. · 15–55'

Claim	Significado	Valor en nuestro token
`iss`	Emisor (issuer)	api-fullstack
`sub`	Sujeto: id del usuario	1
`iat`	Emitido en (issued at)	1793742337 → 16:45 h
`exp`	Expira en (expiration)	1793743237 → 17:00 h (15 min)
`aud · nbf · jti`	Audiencia · no antes de · id único	opcionales
`nombre · rol · amr`	Claims propios (privados)	Ana · admin · ["pwd"]



Fechas en segundos Unix

iat y exp cuentan segundos desde 1970-01-01 UTC. jwt.verify() compara exp con la hora actual y rechaza el token vencido.

```
exp - iat = 900 s
```

¿Cómo se firma? HS256 vs. RS256

TRANSF. · 15–55'

```
firma = HMACSHA256( base64url(header) + "." + base64url(payload), JWT_SECRET )
// Si cambias 1 carácter del payload, la firma recalculada ya no coincide
```



HS256 · simétrico

- Una sola clave secreta firma y verifica.
- Ideal cuando un único back-end emite y valida (nuestro demo).
- Riesgo: quien tenga la clave puede fabricar tokens.

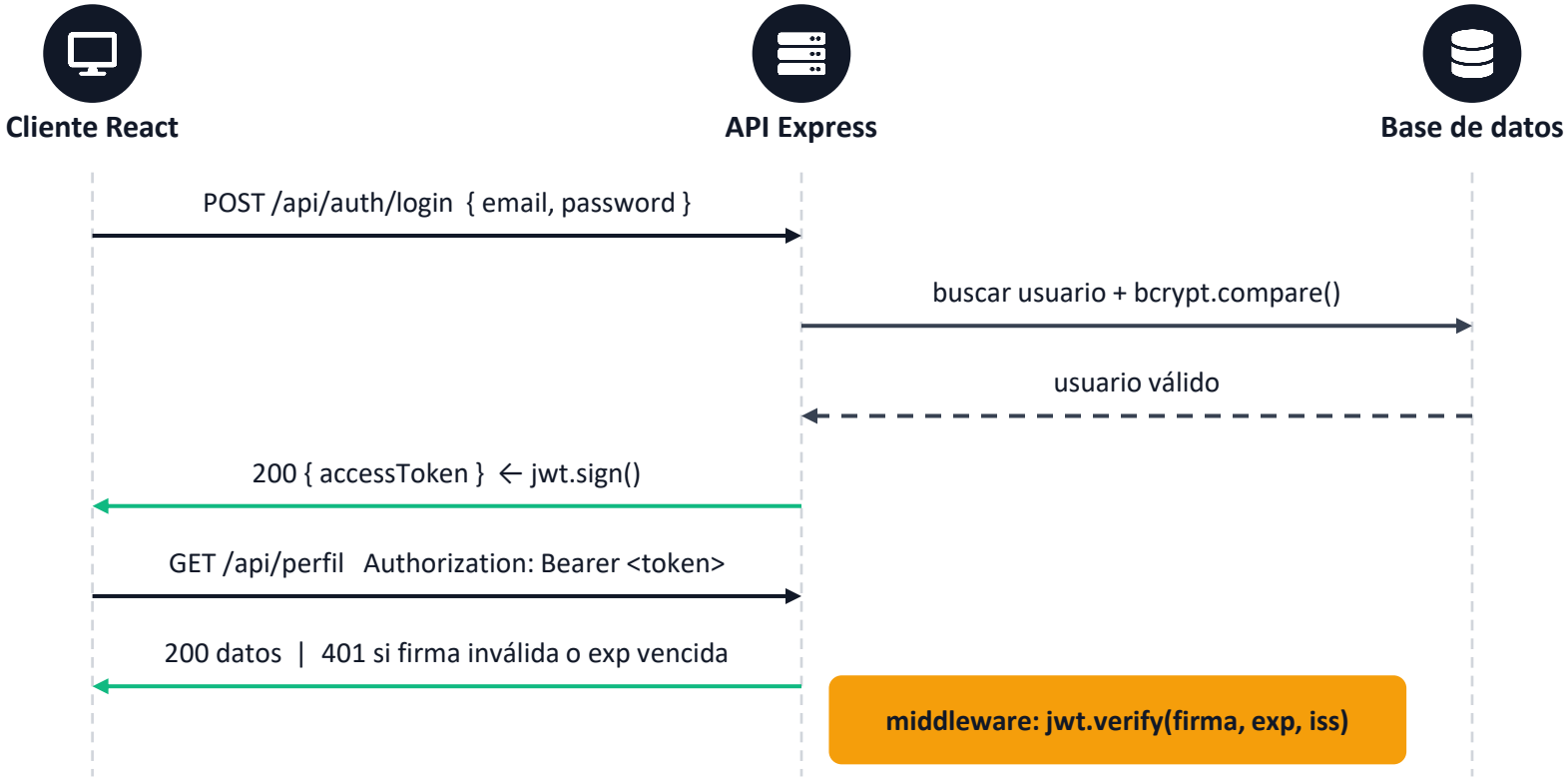


RS256 · asimétrico

- Clave privada firma; clave pública verifica.
- Ideal para microservicios y proveedores (Google, Auth0, Keycloak).
- Los servicios solo necesitan la clave pública.

Flujo de autenticación con JWT

TRANSF. · 15–55'



Código 1 · Emitir el token en el login

TRANSF. · 15–55'

server.js

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const JWT_SECRET = process.env.JWT_SECRET;

function firmarAccessToken(u) {
  return jwt.sign(
    { sub: u.id, nombre: u.nombre, rol: u.rol },
    JWT_SECRET,
    { algorithm: 'HS256', expiresIn: '15m',
      issuer: 'api-fullstack' });
}

app.post('/api/auth/login', (req, res) => {
  const { email, password } = req.body;
  const u = buscarPorEmail(email);
  if (!u || !bcrypt.compareSync(password, u.hash))
    return res.status(401)
      .json({ error: 'Credenciales inválidas' });
  res.json({ accessToken: firmarAccessToken(u) });
});
```

Mismo mensaje de error
si falla el email o la contraseña: no revela qué usuarios existen.

expiresIn: 15m
limita el daño si el token es robado.

La clave va en .env
nunca en el código ni en GitHub.

Código 2 · Middlewares de seguridad

TRANSF. · 15–55'

server.js · middlewares

```
// AUTENTICACIÓN: ¿quién eres?
function verificarToken(req, res, next) {
  const [tipo, token] = (req.headers.authorization || '').split(' ');
  if (tipo !== 'Bearer' || !token) return res.status(401).json({ error: 'Token requerido' });
  try {
    req.usuario = jwt.verify(token, JWT_SECRET, { algorithms: ['HS256'], issuer: 'api-fullstack' });
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Token inválido o expirado', detalle: err.message });
  }
}
// AUTORIZACIÓN: ¿qué puedes hacer?
const autorizar = (...roles) => (req, res, next) =>
  roles.includes(req.usuario.rol) ? next() : res.status(403).json({ error: 'Acceso denegado' });

app.get('/api/perfil', verificarToken,
  (req, res) => res.json({ mensaje: `Hola ${req.usuario.nombre}` }));
app.get('/api/admin/reportes', verificarToken, autorizar('admin'),
  (req, res) => res.json({ reportes: [] }));
```

Resultados · Login y acceso a rutas protegidas

TRANSF. · 15–55'

Thunder Client · login

```
$ POST /api/auth/login
{ "email": "ana@uni.edu.pe",
  "password": "Ana12345" }
→ 200 OK
{ "requiere2FA": false,
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5…" }

$ POST /api/auth/login  (password: "xxx")
→ 401 Unauthorized
{"error":"Credenciales inválidas"}
```

Thunder Client · /api/perfil

```
$ GET /api/perfil   (sin header)
→ 401 Unauthorized
{"error":"Token requerido"}

$ GET /api/perfil
  Authorization: Bearer eyJhbGci…
→ 200 OK
{ "mensaje": "Hola Ana",
  "rol": "admin",
  "metodos": ["pwd"] }
```

Resultados · ¿Qué pasa si intentan hacer trampa?

TRANSF. · 15–55'

Token manipulado

Se cambió en el payload nombre: “Hacker” y se reutilizó la firma original.

```
→ 401 Unauthorized
"detalle": "invalid signature"
```

Token expirado

Token firmado con exp 60 segundos en el pasado.

```
→ 401 Unauthorized
"detalle": "jwt expired"
```

Rol insuficiente

Luis (estudiante) pide /api/admin/reportes con un token válido.

```
→ 403 Forbidden
"error": "Acceso denegado:
 se requiere rol admin"
```

Del lado del front-end: React consume la API

TRANSF. · 15–55'

frontend · fetch

```
// src/api/auth.js
const API = 'http://localhost:3000/api';

export async function login(email, password) {
  const r = await fetch(`${API}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password }) });
  return r.json();
}

export async function getPerfil(token) {
  const r = await fetch(`${API}/perfil`, {
    headers: { Authorization: `Bearer ${token}` } });
  if (r.status === 401) throw new Error('Sesión expirada');
  return r.json();
}
```

¿Dónde guardo el token?

localStorage
Fácil, pero lo lee cualquier script inyectado (XSS).

Estado en memoria (Zustand)
Más seguro; se pierde al recargar la página.

Cookie httpOnly + Secure
No accesible desde JS. Recomendado en producción (cuidar CSRF).

Parte 2 · Autenticación en dos pasos

Factores de autenticación, TOTP y su integración con JWT · 35–55 min



Los tres factores de autenticación

TRANSF. · 15–55'



Algo que sabes

Contraseña, PIN, patrón



Algo que tienes

Celular con app autenticadora, llave física



Algo que eres

Huella, rostro, voz

2FA = dos factores de categorías DISTINTAS. Contraseña + pregunta secreta NO es 2FA: ambos son “algo que sabes”. Si roban tu contraseña, el atacante aún necesita tu celular.

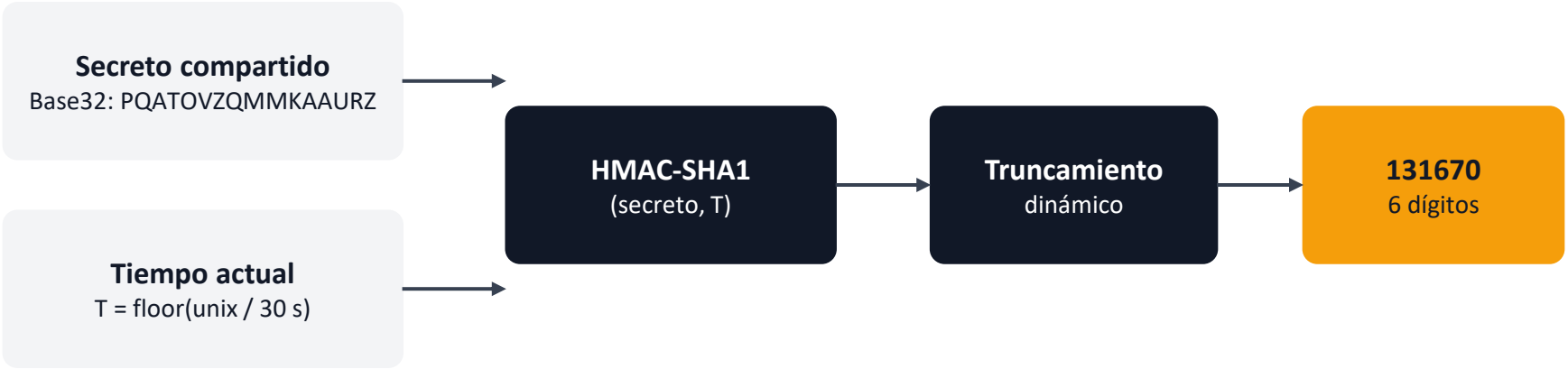
Métodos de 2FA: seguridad vs. comodidad

TRANSF. · 15–55'

Método	Seguridad	Comodidad	Observación
Código por SMS	Baja-media	Alta	Vulnerable a SIM swapping e interceptación
Código por correo	Media	Media	Depende de la seguridad del correo
App TOTP (Google/Microsoft Authenticator)	Alta	Media	Funciona sin internet · lo implementamos hoy
Notificación push	Alta	Muy alta	Riesgo de “fatiga de MFA” (aprobar por cansancio)
Llave FIDO2 / passkeys	Muy alta	Alta	Resistente a phishing

¿Cómo funciona TOTP? (RFC 6238)

TRANSF. · 15–55'



El secreto se comparte una sola vez
mediante el código QR al activar 2FA.

Cada 30 segundos cambia T
y por lo tanto cambia el código.

App y servidor calculan lo mismo
sin comunicarse: si coinciden, ✓ válido.

Activar 2FA: secreto, QR y primer código

TRANSF. · 15–55'



QR generado por la API (escanéalo con tu app autenticadora en el laboratorio)

resultado real · activación

```
$ POST /api/2fa/configurar   (Bearer token de Ana)
→ 200 OK
{ "secreto": "PQATOVZQMMKAAURZ",
  "otpauth": "otpauth://totp/API%20FullStack%20UNI:
      ana%40uni.edu.pe?secret=PQATOVZQMMKAAURZ
      &period=30&digits=6&algorithm=SHA1",
  "qrDataUrl": "…" }

// La app autenticadora muestra: 131670

$ POST /api/2fa/activar  { "codigo": "131670" }
→ 200 OK
{ "mensaje": "2FA activado correctamente" }
```

Login en dos pasos combinado con JWT

TRANSF. · 15–55'

1 **Paso 1 · contraseña**

`POST /api/auth/login`

tokenTemporal
scope: 2fa-pendiente · 5 min

→

2 **Paso 2 · código TOTP**

`POST /api/auth/2fa/verificar`

{ tokenTemporal, codigo }

→

✓ **Token completo**

`accessToken`

amr: ["pwd", "otp"] · 15 min

Clave del diseño: el token temporal solo sirve para el paso 2. Si alguien lo usa en una ruta protegida, el middleware lo rechaza. Así una contraseña robada no abre la API.

Código 3 · Verificar el segundo paso

TRANSF. · 15–55'

server.js · 2FA

```
const { authenticator } = require('otplib');

// Paso 1 (login): si el usuario tiene 2FA, NO entregamos el accessToken
if (u.dosPasos) return res.json({ requiere2FA: true, tokenTemporal: firmarTokenTemporal2FA(u) });

// Paso 2: validar token temporal + código de 6 dígitos
app.post('/api/auth/2fa/verificar', (req, res) => {
  const { tokenTemporal, codigo } = req.body;
  try {
    const p = jwt.verify(tokenTemporal, JWT_SECRET, { algorithms: ['HS256'] });
    if (p.scope !== '2fa-pendiente') throw new Error('Token no válido para 2FA');
    const u = buscarPorId(p.sub);
    if (!authenticator.verify({ token: codigo, secret: u.totpSecret }))
      return res.status(401).json({ error: 'Código 2FA incorrecto' });
    res.json({ accessToken: firmarAccessToken(u) });   // amr: ['pwd', 'otp']
  } catch (err) { res.status(401).json({ error: err.message }); }
});
// En verificarToken(): if (payload.scope === '2fa-pendiente') → 401
```

Resultados · Login con 2FA activado

TRANSF. · 15–55'

resultado real · intentos

```
$ POST /api/auth/login  (Ana, contraseña OK)
→ 200 OK
{ "requiere2FA": true,
  "tokenTemporal": "eyJhbGciOiJIUzI1Ni…" }

$ GET /api/perfil  (Bearer tokenTemporal)
→ 401 Unauthorized
{ "error": "Completa el segundo paso (2FA)" }

$ POST /api/auth/2fa/verificar  codigo: "000000"
→ 401 Unauthorized
{ "error": "Código 2FA incorrecto" }
```

resultado real · éxito

```
$ POST /api/auth/2fa/verificar
  { tokenTemporal, codigo: "<app>" }
→ 200 OK
{ "accessToken": "eyJhbGciOiJIUzI1Ni…" }

$ GET /api/perfil  (Bearer accessToken)
→ 200 OK
{ "mensaje": "Hola Ana",
  "rol": "admin",
  "metodos": ["pwd", "otp"] }
```

Buenas prácticas de seguridad

TRANSF. · 15–55'

Secreto fuerte en .env

Mínimo 256 bits aleatorios; nunca en el repositorio.

Fija el algoritmo

algorithms: ['HS256'] al verificar evita el ataque alg:none.

Tokens de vida corta

Access token de 15 min + refresh token para renovar.

Payload sin secretos

Es legible por cualquiera: solo id, rol y lo mínimo.

HTTPS siempre

Un token interceptado en HTTP equivale a una sesión robada.

Limita intentos de 2FA

Rate limiting y códigos de respaldo de un solo uso.

Laboratorio guiado · 30 minutos

PRÁCTICA · 55–85'

5' **Instalar y levantar**
Descomprimir demo-jwt-2fa.zip → npm install → npm start

5' **Login y análisis del token**
Thunder Client o Postman: POST /api/auth/login; pegar el token en jwt.io

5' **Autorización por roles**
GET /api/admin/reportes con Luis (403) y con Ana (200)

10' **Activar 2FA con tu celular**
POST /2fa/configurar → abrir qrDataUrl en el navegador → escanear → /2fa/activar

5' **Login en dos pasos y expiración**
Completar el paso 2; luego cambiar expiresIn a '1m' y observar "jwt expired"

README

```
# Usuarios de prueba

ana@uni.edu.pe
Ana12345
rol: admin

luis@uni.edu.pe
Luis12345
rol: estudiante

$ npm start
$ npm run demo
```

Reto para el avance del proyecto y rúbrica

PRÁCTICA · 55–85'

Reto

Integra en el proyecto del curso: login con JWT, rutas protegidas por rol y 2FA con TOTP. La pantalla de login en React debe pedir el código cuando la API responda requiere2FA: true.

Criterio	Puntos
Login emite JWT firmado con exp e iss	4
Middlewares: 401 sin token o inválido; 403 por rol	5
2FA TOTP: activación con QR y verificación	6
Front-end React envía Bearer y maneja 401	3
Buenas prácticas: .env, algorithms, sin datos sensibles	2
Total	**20**

Retroalimentación y metacognición

CIERRE · 85–90'

Un estudiante con token válido pide una ruta de admin. ¿Qué código devuelve la API?
403 Forbidden

¿Por qué el token manipulado devolvió “invalid signature”?
La firma ya no coincide con header + payload

¿Para qué sirve el token temporal con scope 2fa-pendiente?
Solo habilita el paso 2, no las rutas

Respuestas esperadas en celeste: revelarlas después de que el grupo responda.



Reflexiona

- ¿Qué parte del flujo te costó más entender?
- ¿Qué cambiarías en tu proyecto después de hoy?
- ¿Activarías 2FA en tus cuentas personales?

Ideas para llevarse

Próxima sesión: Retroalimentación del avance del proyecto

JWT firmado (JWS) = identidad verificable sin estado; se lee, pero no se puede alterar.

Autenticación responde 401; autorización por roles responde 403.

2FA con TOTP: si roban la contraseña, aún falta el celular.